

Group 18

Jason Bi, Pranav Shukla, Prithvi Saha

IMU infused 2-Dimensional Lidar

Mechatronics Project Report

for

Mechatronics (ROB-GY 5103)

Instructed by
Prof. Vikram Kapila



Abstract

The following report proposes a software stabilized 2D Lidar system specifically designed for indoor navigation applications. The primary objective is to build a high-utility, cost-effective functional solution for mapping and navigation of indoor spaces that is resistant to perturbations. There is an emphasis on minimizing cost, size, weight, and time of measurement for the system.

We accomplish a noise resistant 2-D Lidar system, and a Manual 3-D mapping mode for the sensor as well.

INDEX

1. Abstract.....	
2. Lidar Systems, State of the Art, and Applications.....	
• What are Lidar Systems?	
• Importance of 2D Lidar Systems	
• Applications of 2D Lidar Systems in Indoor Environments	
• State of the Art	
3. Design Blueprint.....	
• Base Plate	
• Gear Encoder	
• Top Plate Clamp	
• BS2 Board of Education Holder	
4. Components Selection.....	
5. Prototyping Process.....	
• Mechanical Prototyping	
• Finalized Mechanical Design	
6. Electrical Design.....	
• Circuit	
• Safety Considerations	
7. Signal Architecture.....	
8. User Interfacing and Data Visualization.....	
9. Future Improvements.....	
10. Conclusion.....	
11. Appendix.....	
• Bill of Materials	
• Production Cost	
• Mathematical Computations	
• Programs	
• References	

Lidar Systems, State of The Art and Applications

What are Lidar Systems?

Lidar (Light Detection and Ranging) systems use laser technology to measure distances by emitting light pulses and analyzing their reflections. These systems create detailed spatial representations of environments, either in 2D or 3D.

2D Lidar, in particular, operates on a single plane (usually horizontal) and provides two-dimensional data for mapping and object detection. It is widely used in indoor applications due to its simplicity, cost-effectiveness, and efficiency.

Importance of 2D Lidar Systems

- **Accuracy:** Provides precise measurements for navigation and obstacle detection.
- **Cost-Effectiveness:** More affordable than 3D Lidar and other Point Cloud Generating Sensors, making it accessible for various industries.
- **Reliability:** Unlike Visible Spectrum Sensing Systems, LIDAR Performs well under diverse environmental conditions, such as darkness or changing light levels
- **Real-Time Processing:** Enables quick data collection and analysis, crucial for applications like robotics.

Applications of 2D Lidar Systems in Indoor Environments

- **Indoor Mapping:** Creates accurate maps for navigation and facility management.
- **Robotics:** Enables autonomous navigation, obstacle avoidance and SLAM
- **Security & Surveillance:** Tracks movement and monitors restricted areas.
- **Augmented Reality (AR):** Provides spatial awareness for enhanced AR experiences.
- **Healthcare & Elderly Monitoring (Gerontechnology) / Intrusion Detection:** Detects falls or intrusions and monitors activities for safety.

State of the Art

- **Garmin LIDAR-Lite v3HP (\$150):** Offers accurate distance readings with a compact design suitable for various applications³.
- **TeraRanger Evo 60m USB ToF Rangefinder (\$118):** Provides a long range with a lightweight and compact form factor³.
- **For 3D systems, Basic Models (\$10,000 to \$20,000):** Entry-level models suitable for general applications⁶.
- **High-End Models (Luminar, Velodyne) (up to \$80,000):** Advanced models offering high resolution and range for specialized applications⁶.

In summary, 2D Lidar systems are transforming indoor environments by enabling precise mapping, navigation, and activity monitoring across healthcare, robotics, logistics, and more. Their affordability and reliability make them indispensable tools for modern applications.

Design Blueprint

The 3D Lidar housing is composed of the Base Plate, Gear Encoder, and Top Plate Clamp.

- The **Base Plate** houses all the electronics of the system.
- The **Gear Encoder** rotates the mirror which translates the 1D Lidar to 3D as well as spins along the Base Plate, not with.
- The **Top Plate Clamp** holds the Gear Encoder in place to prevent vertical slippage.
- The **BS2 Board of Education Holder** houses the BS2 Board so that's an all-in-one system

Base Plate (Refer to Fig 2.1)

- The motor, IMU, and IR sensors are mounted to the base plate. There are cutouts and erected structures in the plate that allow for the electronics to be slotted/fastened down to.
- The electronics are connected to the BS2 and power via wires that are fed out from a hole in the base plate.
- The motor is attached to a small gear that translates the motion to the larger encoder gear that is mounted directly adjacent to it. The gear ratio is approximately 1:3, but the main purpose was to find a very tight fit to avoid slippage. Alignment in both the x and y axis were critical to allow for a smooth rotation.
- The IR sensors are able to detect the rotation in the gear encoder which allows us to keep track of the angle as we rotate.
- The IMU kept track of the acceleration which is pivotal for calibration.
- The LIDAR is mounted in the center of the Base Plate and is reflected off a mirror that sits directly above it at a 45 degree tilt. This is what translates the LIDAR to 2D.

Gear Encoder (Refer to Fig 2.2)

- The Gear Encoder has a thin layer of markings that allow the IR sensors to determine the angular position during a 360 degree run.
- The Gear Encoder sits flush against a groove in the Base Plate that allows it to spin without the rest of the system's housing moving.
- The Gear Encoder has a layer of teeth along the inside of it that allow the motor gear (Fig 2.6) to translate the rotation to it.
- The Mirror Mount (Fig 2.5) is glued down concentrically with the LIDAR hole in the middle

Top Plate Clamp (Refer to Fig 2.3)

- The Top Plate Clamp screws into the Base Plate to effectively “locks” the gear encoder in place vertically, stopping it from shifting during rotation.

BS2 Board of Education Holder (Refer to Fig 2.7)

- This piece sits below the Base Plate as houses all the connections and wires as well as the BS2 Board of Education itself
- There is a spacer component (Fig 2.8) that stops any of the wires from being crumpled or the Base Plate from sitting directly on the board.

Components Selection

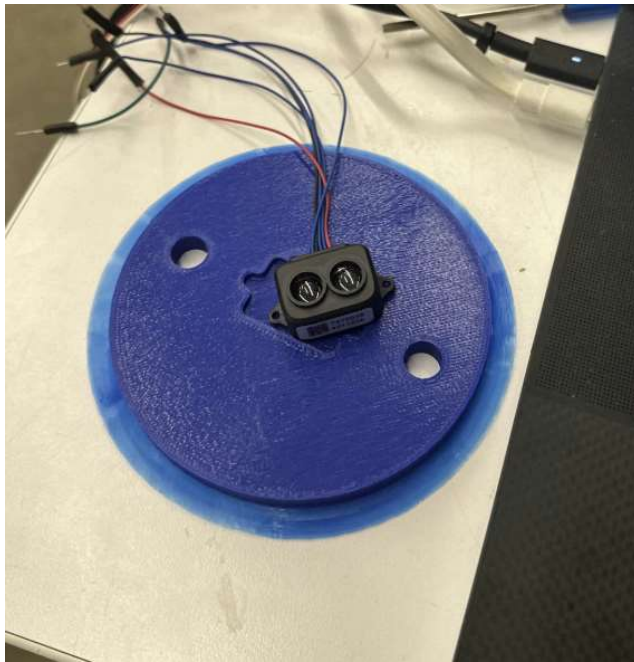
Component	Consideration	Chosen	Ideal
Primary uC	Single Core,Multi Core, Interrupts, Cost	BS2	Interrupt Based uC
Actuator	Small Size, High Speed, Optimum Torque, Power consumption, Feedback	Parallax Continuous Servo	Geared N20.
Encoders	High Accuracy, Reliability, Size constrained	2- IR pairs with concentric black/white slits	IR array with Absolute Optical Encoders
TOF Sensor	Accuracy, Range, Cost	TF-Luna (8-m)	Building sensor with TOF IC and laser e/r
IMU	Accuracy, Inbuilt Filtering, Cost, Reliability	Nano Sense	WT1-IMU ICM42607
Chassis	Size, Shaft, and Cost	3d Printed Hollow Shaft Mount	IDEAL
Power Regulator	Cost, Reliability, Current tolerance	Zener Resistor Regulator	Voltage Regulator IC

Prototyping Process

Mechanical Prototyping

For prototyping, a majority of our mechanical problems came from dimensional constraints and assembly problems. Issues arose especially for the mounts and gears because we had to account for shifting, and slippage. The resolution of the 3D printers in the Makerspace would often lead to problems if we didn't include substantial tolerance values. There were also a lot of moving components that required calibration testing which would influence to change our "Base Plate" design. This part had the most reprints as we checked the fitting of components during reprints and would build/modify previous designs depending on fittage as well as if we had any design oversights.

Below are renditions of our designs for the Base Plate:



In this version of the prototype, we tested a mold to mount the LIDAR in. This Base Plate design was minimal and we had to make sure the LIDAR sat flushed against the surface. This is when we first encountered that we had to leave tolerances on all our prints because using exact dimensions made the fit way too tight/impossible.

Fig 1.1 Lidar Base Plate



Fig 1.2 Base Plate with Holes for Clamp

In this design of the Base Plate, we made it compatible with a lid that would be inserted along the other ring. However, we had a design flaw in which we could not isolate the rotation of the mirror from the LIDAR sensor (ie: the mirror should rotate above the LIDAR, but the LIDAR should not rotate with the mirror). The design oversight made this part unusable, but was the first step to getting a working prototype.



Fig 1.3 First Working Base Plate

This design of the Base Plate was had some tolerance issues with the running of wires out of the mold and the mounting of the encoders/motor. However, we used a dremel to remove material and slot the components in and were able to test on this design. It was our first prototype in which we were able to start integrating electronics. Eventually, we would redesign this part with cutouts so that every component sat well and wires wouldn't tangle.



From left to right we have our oldest to newest prototypes. The left-most gear was the most primitive design without the incorporation of encoders. The middle-most gear was an attempt to have 360 slits to be read by the IR sensors, however those lines were way too thin to be picked up on. For the final prototype, we tried thicker lines and 90 slits for a higher resolution, but the IR sensors were still missing a lot of readings.

Fig 1.4 Gear Encoder Prototypes



The clamp on the bottom was for Fig 1.2 prototype base plate, and we never ended up using it as the design was not working.

The clamp on the top was used for our final design (Fig 2.1), but was too flimsy so we had to reprint it thicker.

Fig 1.5 Top Clamp Prototypes



The top mirror mount was our initial design. We wanted to directly connect the motor to rotate the mirror, however we did not consider how to isolate the rotation. For the bottom mirror mount, it was designed in accordance to the base plate in Fig 1.3, and it worked well rotationally. However, the teeth design had some slippage and we ultimately just had to reprint one with better resolution.

Fig 1.6 Motor Mount Prototypes



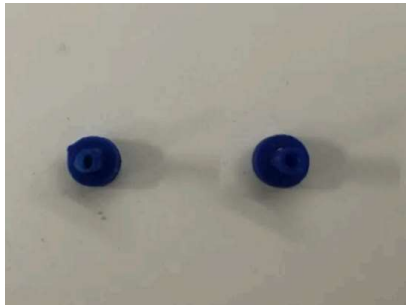
These motor mounts were designed for a motor with a built-in encoder (feedback) that we bought and planned to use instead of the IR sensors and a printed encoder. We scratched the idea ultimately because the manufacturing of the motor with the built-in encoder was low quality and had many problems with current and gear translation.

Fig 1.7 Motor with Encoder Mount



We went through several prints and renditions of motor gear designs which we would mount to the Parallax Continuous Rotation Servo Motor. The main reason we had to print several versions is because even the slightest slippage or teeth misalignment would force us to adjust and reprint.

Fig 1.8 Motor Gear Prototypes



These were also designed for the motor with a built-in encoder. They were meant to be use to hold the gear at the right height.

Fig 1.9 D-Shaft Gear Spacer

Finalised Mechanical Design

Attached below are screenshots of the mechanical components (Base Plate, Gear Encoder, Top Plate Clamp, Motor Shaft, Motor Gear, and Mirror Mount). In each figure the component features are explained.

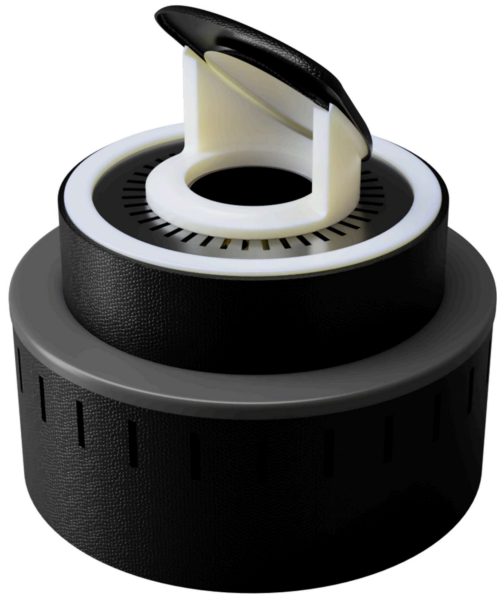


Fig. 2.1.1 Final Design Render

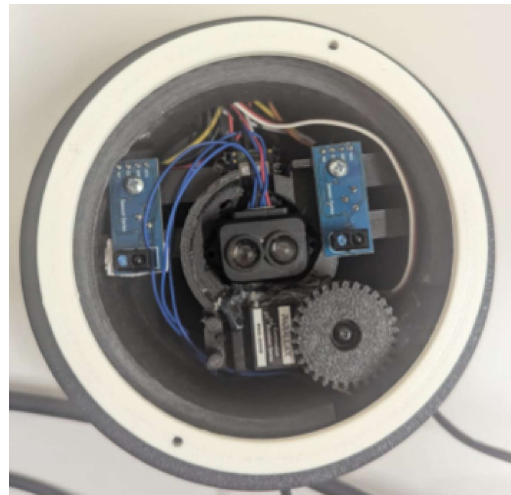


Fig 2.1.1.2: Internal Assembly

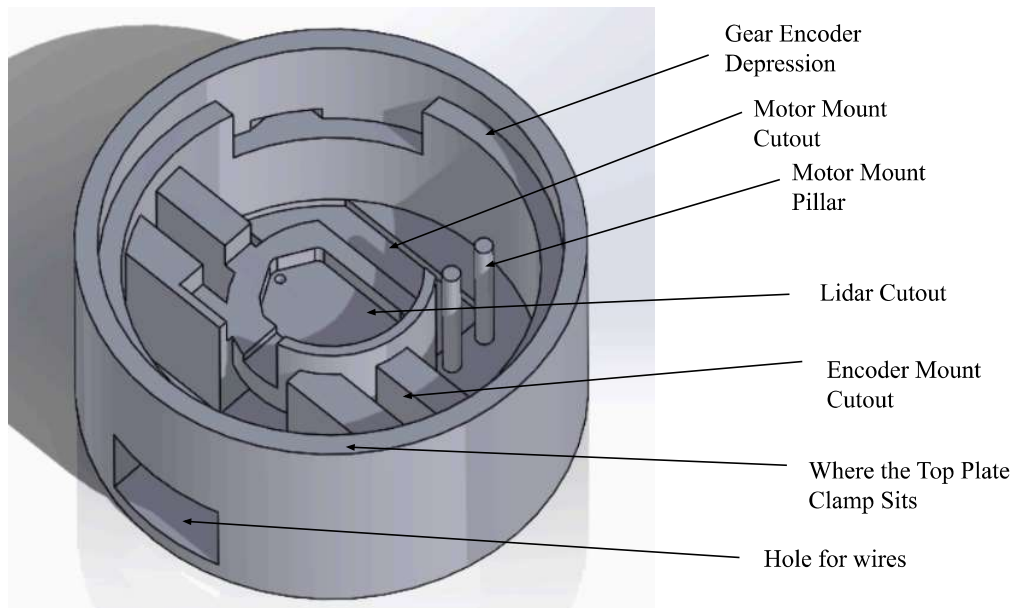


Fig 2.1.2 Base Plate

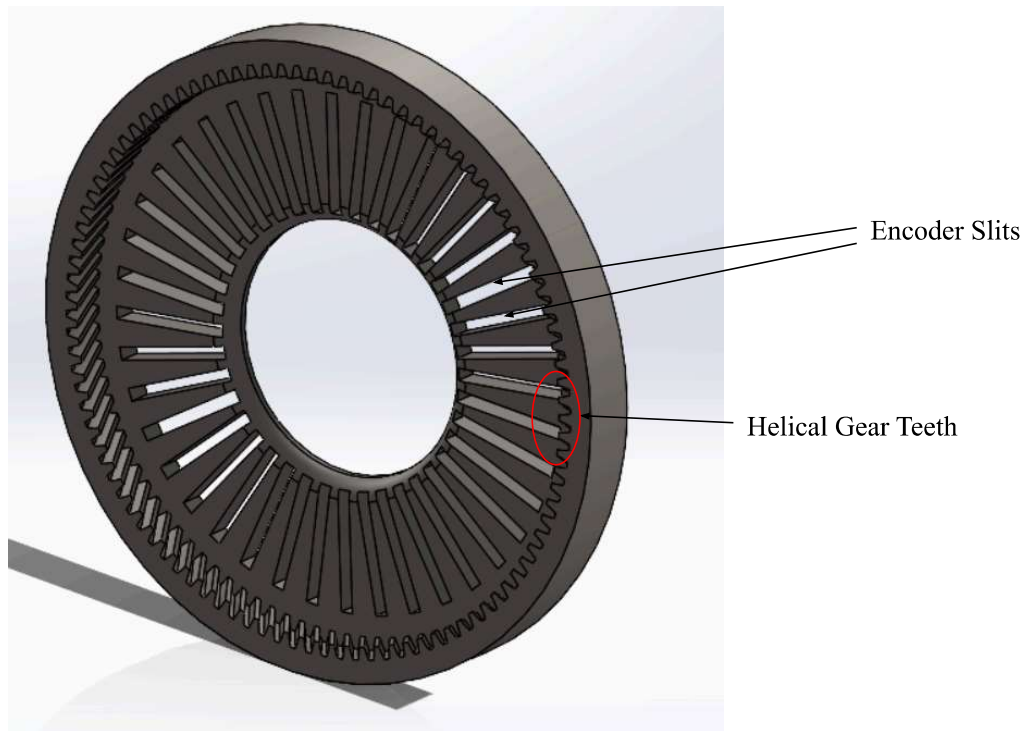


Fig 2.2 Gear Encoder

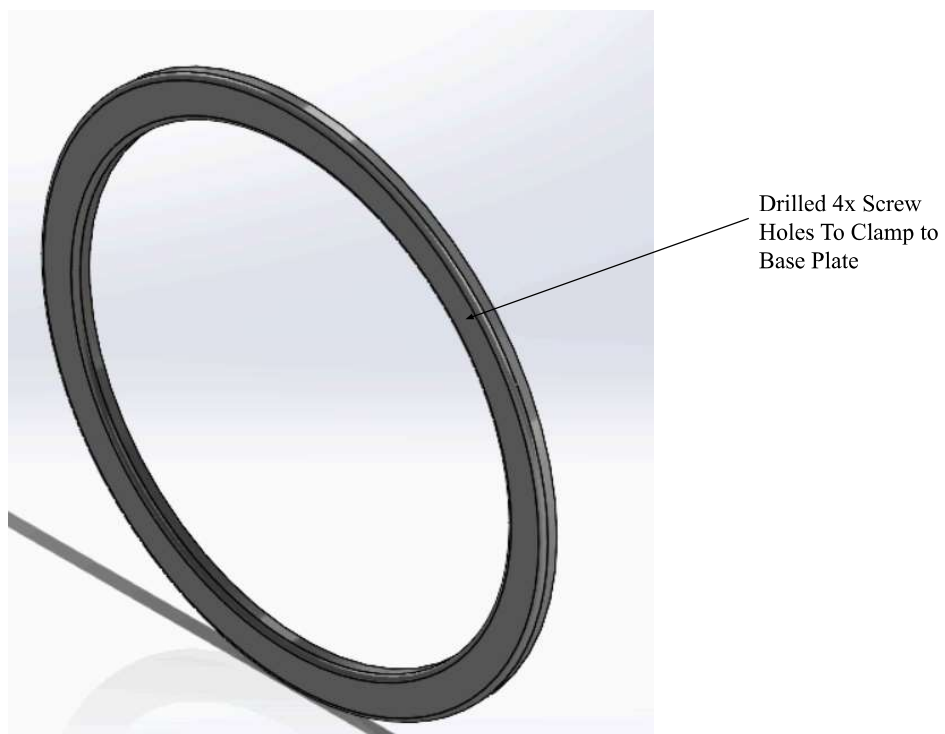
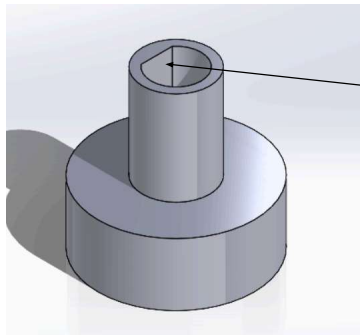
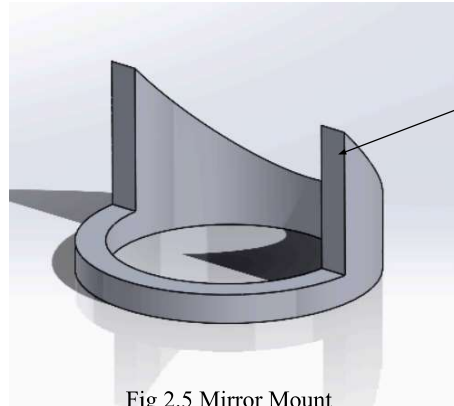


Fig 2.3 Top Plate Clamp



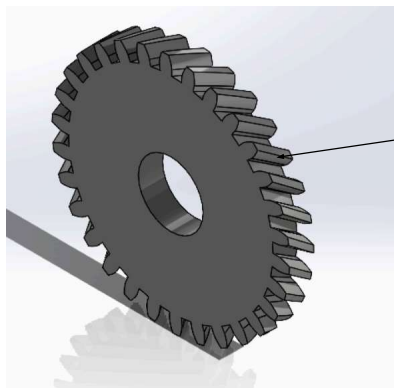
D-shaft to sit on motor to lock

Fig 2.4 Motor Shaft Spacer



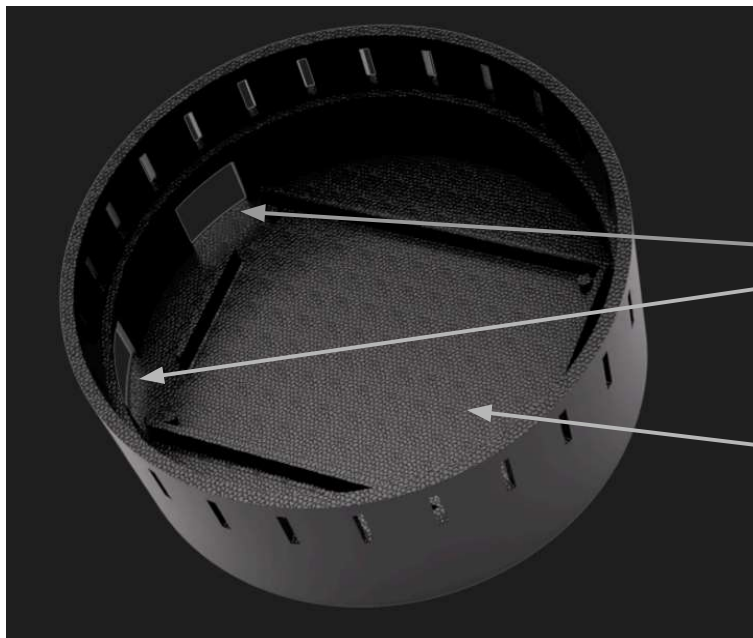
Mirror Mount
(45deg)

Fig 2.5 Mirror Mount



Helical Gear Teeth
(1:3 Ratio with
Gear Encoder)

Fig 2.6 Motor Gear



Holes for running
power and com port

BS2 Cutout

Fig 2.7 BS2 Board of Education Holder

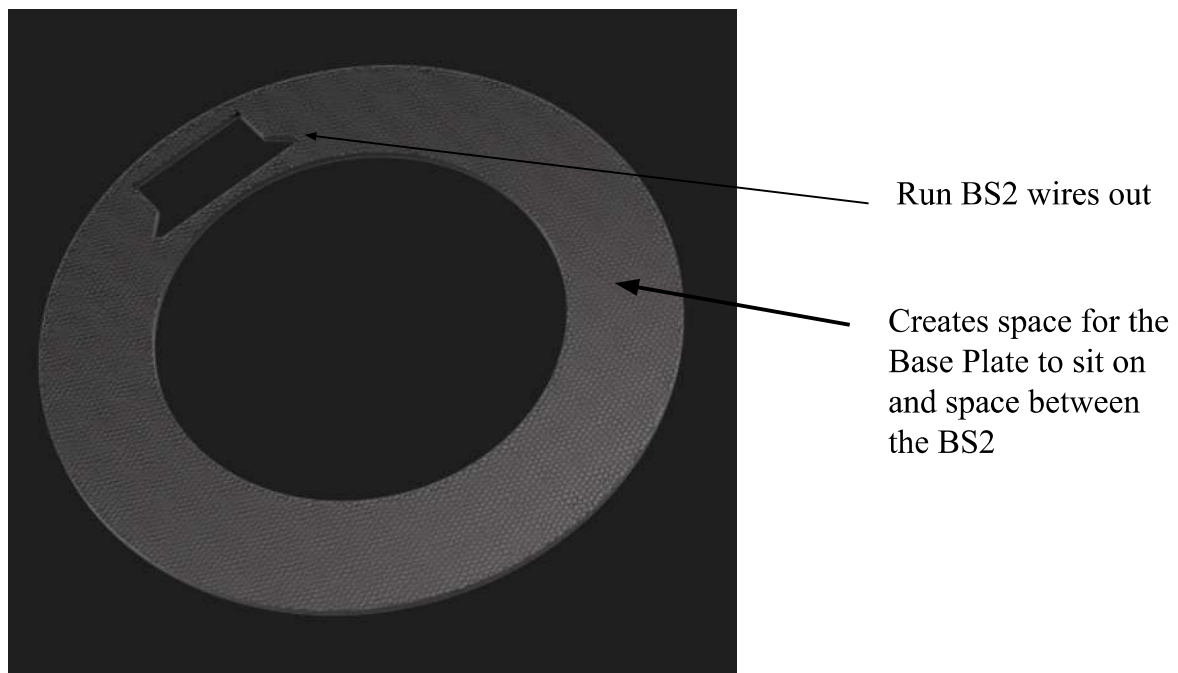


Fig 2.8 BS2 Spacer

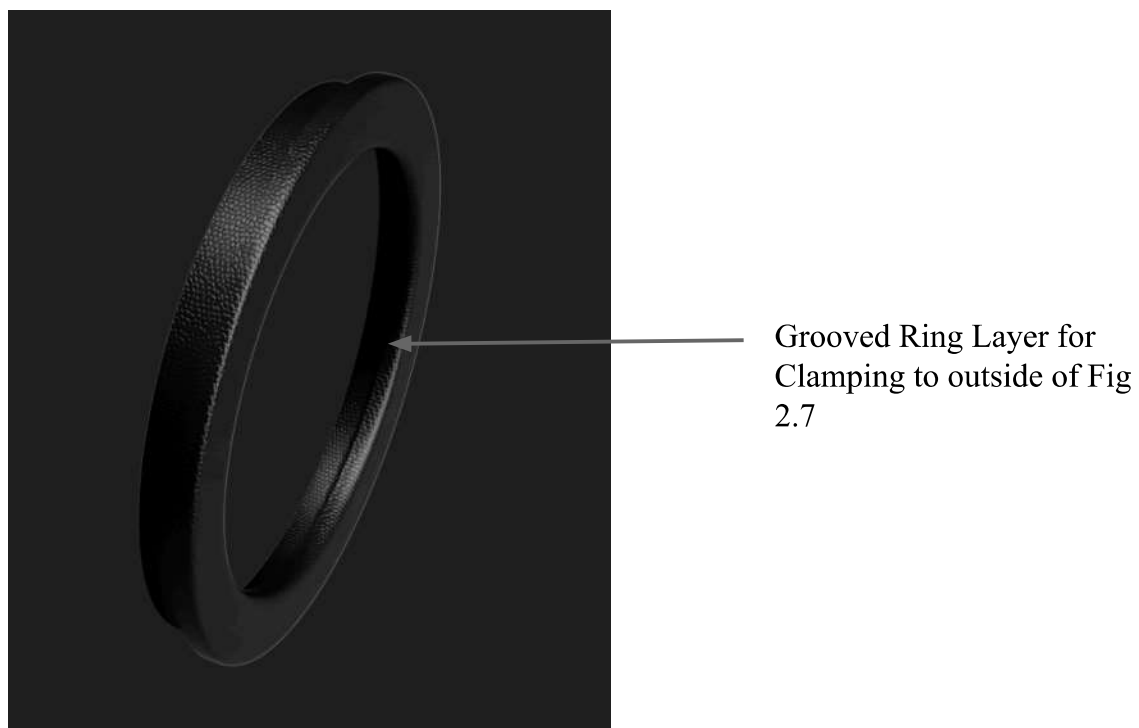
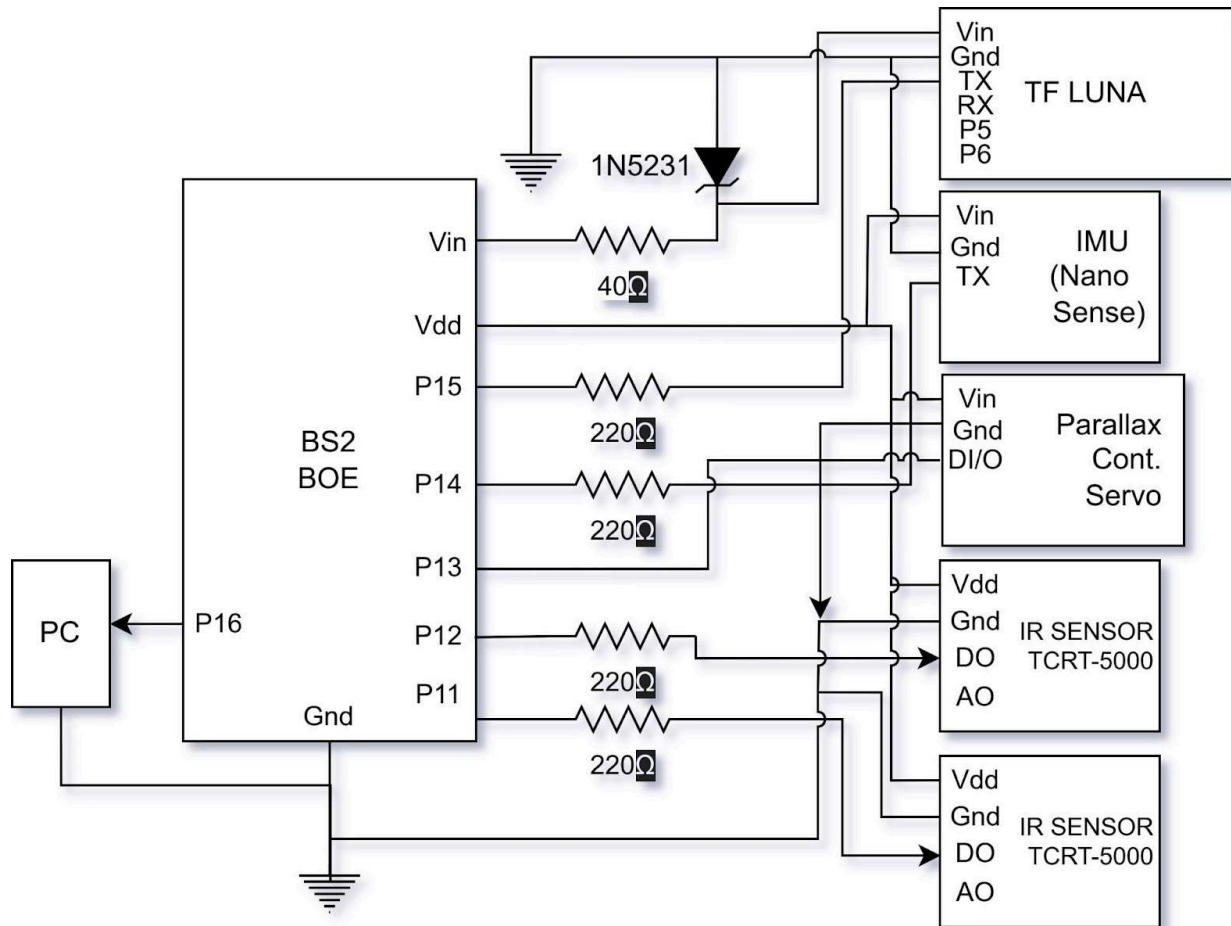


Fig 2.9 BS2 Mount Clamp

Electrical Design

Circuit



Primary Components:

1. Microcontroller: BS2 BOE

- The BS2 BOE microcontroller serves as the central control unit.
- It interfaces with all sensors, the servo motor, and the PC for data processing and control.

2. TF LUNA (1D LIDAR Sensor)

- Connected to the microcontroller via pins P15 (TX)
- Provides distance measurements in one dimension.
- Powered by the Vin and Gnd pins via the Zener Regulator.

3. IMU (Nano Sense)

- Connected to the microcontroller via a single communication line (TX to P14).
- Used for accelerometer and Kalman Filter Gyro Reading
- Powered through Vdd and Gnd.

4. Parallax Continuous Servo Motor

- Connected to the microcontroller via pin P13.
- Controls the rotation of the LIDAR mirror to achieve 2D scanning.
- Powered by Vdd and Gnd.

5. IR Sensors (TCRT-5000)

- Two IR sensors are connected to pins P11 and P12, respectively.
- Each sensor has three connections:
 - DO (Digital Output): Provides encoder signal to detect slit positions.
 - Vdd and Gnd: Power supply.
- These sensors detect slits on an encoder disk, providing feedback on inner and outer slit positions.

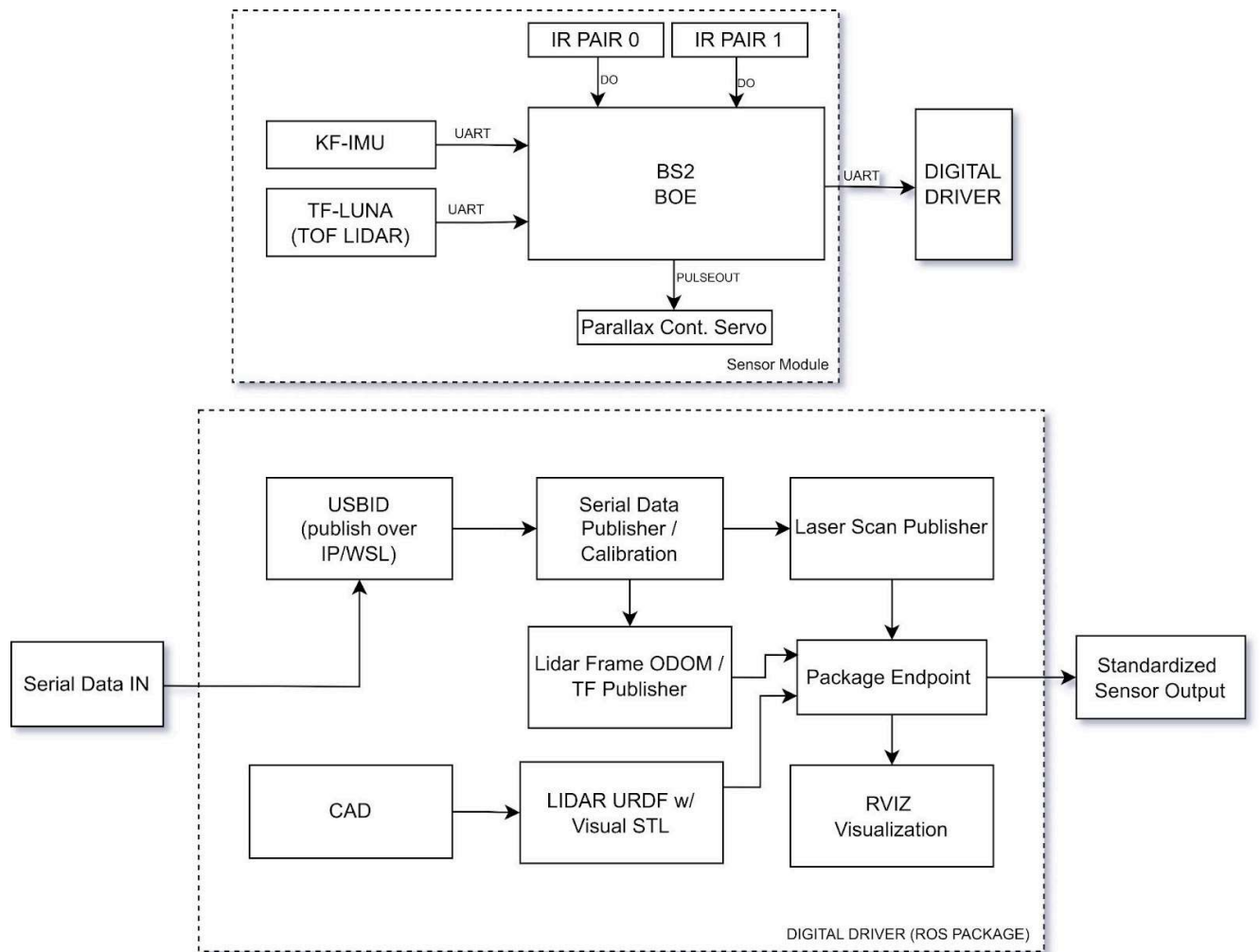
6. Power Supply

- The circuit is powered by a 7V power adapter connected to the BS2 BOE microcontroller.
- A 1N5231 Zener diode and a resistor (40 Ω) are used for voltage regulation and protection of the LIDAR sensor.

Safety Considerations

- *We utilise the Basic Stamp BOE switch as the Kill Switch for the Sensor*
- *We Utilise a Zener Voltage supply to limit max current and max Voltage to the LIDAR sensor.*

SIGNAL ARCHITECTURE



1. Sensor Module

- TF-LUNA (TOF LIDAR) communicates to the BS2 BOE via UART.
- KF-IMU communicates with the microcontroller via UART.
- IR Sensors (TCRT-5000) provide digital output (DO) to the microcontroller.
- Parallax Continuous Servo Motor is controlled by PulseOut signals from the BS2 BOE.

2. Digital Driver (ROS Package)

- Serial Data Publisher/Calibration:
 - ROS NODE that processes serial data from the sensor module and calibrates it for further use.
- Laser Scan Publisher:
 - ROS NODE that converts raw data into a laser scan format suitable for visualization and processing.
- LIDAR Frame ODOM/TF Publisher:
 - ROS NODE that publishes transformation frames and odometry data for stabilisation, and detection of perturbations.
- RVIZ Visualization:
 - Displays the LIDAR scan data in a graphical user interface using ROS's RVIZ tool.
- LIDAR URDF with Visual STL:
 - Provides a Unified Robot Description Format (URDF) model for the LIDAR system, including a visual STL file for simulation or CAD integration.
- USBID (Publish over IP/WSL):
 - Transfers serial data over USB over IP for Windows Subsystem for Linux (WSL) access to serial ports.
- A 1N5231 Zener diode and a resistor (40 Ω) are used for voltage regulation and protection.

Selected Remarks: System/Part Advantages and Disadvantages

Base Plate

- **Advantages**
 - Lots of cutouts and fixtures for parts to be mounted to with good alignment
 - Allows wires and components to flow out easily
 - Electronics can be shifted along some of the mounts for design changes
 - Compact design with no protrusions
 - 3D Printed and cheap
- **Disadvantages**
 - Not a lot of excess space for adding any additional components
 - Motor to gear encoder alignment was difficult to measure out

Hollow Shaft Gear

- **Advantages**
 - Allowed the mirror to rotate without rotating the LIDAR housing, avoiding slip rings.
 - Allows smooth translation of motion among the gear system
 - 3D Printed and cheap
- **Disadvantages**
 - Encoder slits are not able to be read-well with given IR sensors (instead paper was used)
 - Teeth have some slippage due to 3D extruder print resolutions

LIDAR

- **Advantages**
 - Relatively cheap
 - Long-range (8m)
 - Serial communications available
- **Disadvantages**
 - Calibration needed work/irregular values
 - Distance measurements are extremely linear, meaning slight misalignment will give varying values

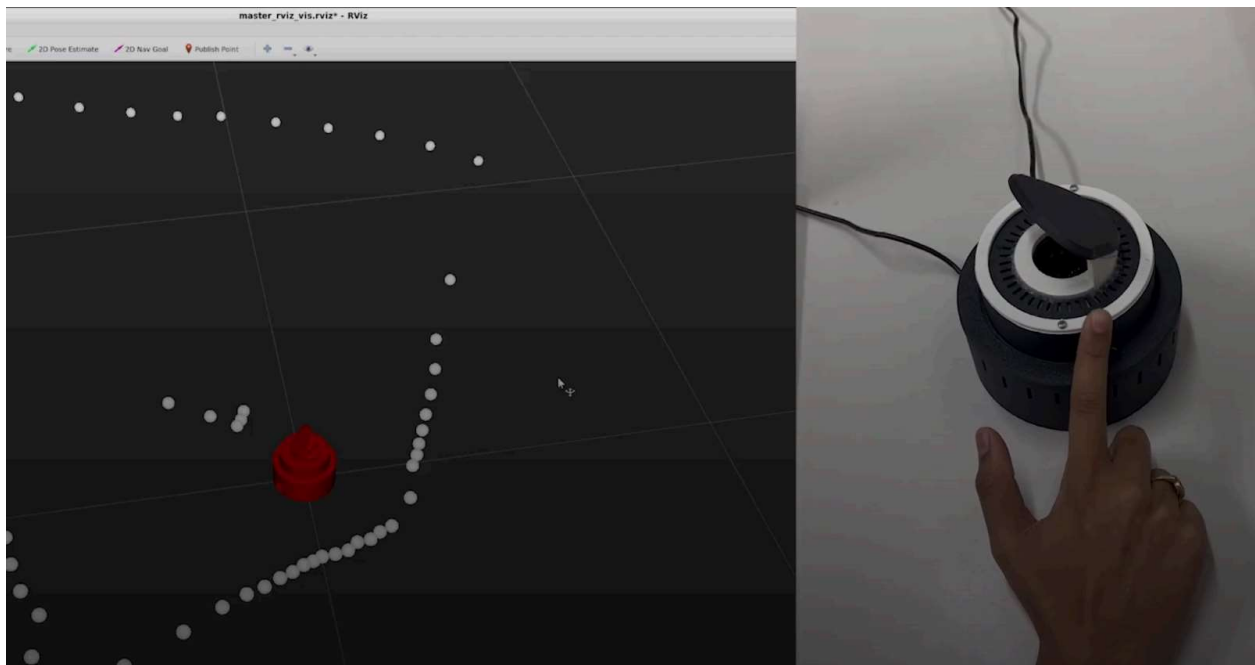
IMU

- **Advantages**
 - Were able to utilize built-in components on Arduino
 - Cheap for our purposes
- **Disadvantages**
 - Configuring the sensors to be read serially was difficult
 - Funnily enough, Accidentally Becomes a Mouse Input, IF SERIAL ERROR occurs.

User Interfacing and Data Visualization

For utilisation of the sensor, a Complete end to end ROS package was developed, which can:

- create a visualization of the LIDAR as well as distance points collected using RVIZ
- Using data from the IMU, offer two modes of utilisation:
 - **3D Modelling:** using shift in Plane of Measurement
 - **Perturbation Monitoring:** By recalculating the Measured distance value in the original plane of measurement.
- PUBLISH an endpoint for Sensor URDF
- PUBLISH odometry data from the sensor in standard ROS odometry format.
- PUBLISH LaserScan data from sensor in Standard format for 2D Lidars



Visualisation in Rviz(left), Final Sensor(right)

Future Improvements

- Improve the resolution of the encoder through enhanced calibration and rigorous testing procedures and a better design.
- Consider using a more economical LIDAR system to significantly reduce overall project costs.
- Further reduce the size of components to achieve a more compact and efficient design.
- Implement long-range calibration capabilities for the LIDAR system to enhance accuracy over extended distances.
- Integrate self-correcting motor speed control using infrared (IR) sensors to improve system performance and responsiveness.

Conclusion

During our project, we learned a lot about the design process steps and the difficulty of combining mechanics and electronics synergistically (Mechatronics). Excluding the discussed issues with tolerances, modeling, and electronics being finicky/uncooperative, we should have approached the project better structurally. A lot of the issues we encountered were due to overestimating how well components would work or thinking that ideas were fine on paper when execution was vastly more difficult. For instance, IR sensors had a hard time picking up encoder slits and the motor would stall if gear ratios weren't perfect.

For future projects, it would be better if we spent more time initially with calibrating sensors and making sure that we have functioning code that reads data values from them, before we begin designing the mechanics. A big part of the reason that we had to keep redesigning parts was because the sensors weren't picking up data or they were extremely variable with the values that were being read. This led us to having to change or reprint parts which would force us to wait for something to finish 3D printing before being able to continue testing

All in all, we were pleased to get the semi-3D Lidar working and being able to reach a final product. There are still some aspects which we wish we had more time to refine, but our project adequately challenged us and made us think of unique approaches in problem-solving.

Appendix

Bill of Materials

Component	Price (\$)	Link
Lidar Sensor	22.09	Amazon
IMU	40.50	Arduino
BS2	49.00	Parallax
Parallax DC Servo Motor	19.95	Parallax
IR Sensor	8.79	Amazon
PLA Filament Roll	13.99	Elegoo (Amazon)

Total Cost

Total cost of raw materials is: $22.09+40.5+49.00+19.95+8.79+13.99= 154.32\$$ (not including tax)

However, we had all the components from the past/mechatronics kit besides the Lidar Sensor and IR Sensor and utilized the makerspace for free components and 3D printing. The only other purchase we made was a motor with a built-in encoder which was 8.77\$, however we scratched the idea as it was too difficult to operate properly. Thus, total out of pocket project-specific cost was $22.09+8.79+8.77= 39.65\$$

Estimated Mass Manufacturing Cost

We estimate that if someone were to build this on their own considering they have their own 3D printer and microcontroller:

- 5\$ for 3D print filament
- 2-3\$ in Solder, Wiring, and Resistors
- 10-20\$ depending on motor quality
- 1\$ for 2 IR sensors
- 12\$ for stand-alone IMU
- 22\$ for LIDAR

Totaling to approximately: **52-63\$**

Considering that manufacturers buy in bulk, we can expect to save up to 40% on parts thus making our total estimated mass manufacturing cost: **31.2\$-37.8\$**

Mathematics and Calibration

- ***IR Sensor Calibration***

To calibrate the IR sensor, we labeled masking tape with distancing from 0-200 cm and measured expected values with actual values that we got. The distance of the IR sensor was measured from the center of the mirror to the face of the object being measured. We utilized multiple points and discovered that the sensor was decently well-calibrated, essentially having a 5cm error between every measurement due to the from the IR sensor to the mirror itself. The further away the item was, the closer the measurements were to the actual value (smaller percent error). However, this is likely due to the object not being exactly perpendicular to the center of the mirror.

- ***Vector State Machine for Encoder:***

We used two IR with colored slits in parallel, one slit with resolution of 180 degree and the other with 45 degree resolution. We designed a Vector State Machine for Computation of the Accurate Angle using Fusion of the two Encoders Given by:

State	Angle	Encoder 0	Encoder 1
1	0-45	0	0
2	45-90	0	1
3	90-135	0	0
4	135-180	0	1
5	180-225	1	0
6	225-270	1	1
7	270-315	1	0
8	315-360	1	1
1	0-45	0	0

We restricted change of a state in one direction, and only one increment at a time. This ensured robustness.

- ***Zener Diode Resistance Selection***

Zener Breakdown Voltage= 5.1V

Zener Power Rating = 500mW

Current Rating = $500/5.1=98.039$ mA

Power Supply Voltage = 7.1V (Measured)

Voltage Drop across Resistor = $8.1-5.1=3V$

Resistance = $V/I = 3/98 * 1000 = 30.612$

For Safety, we choose 40 Ohms as the Zener Resistance.

- **FRAME STABILISATION CALCULATION**

We are computing projections of the distance measured in the inclined (perturbed) plane in the original global reference frame.

Data: We have the perturbed frame is rotated by A_x and A_y with respect to the global frame in fixed angle representation. This gives us the rotation matrix:

$$R = \begin{bmatrix} \cos A_y & 0 & \sin A_y \\ 0 & 1 & 0 \\ -\sin A_y & 0 & \cos A_y \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos A_x & -\sin A_x \\ 0 & \sin A_x & \cos A_x \end{bmatrix}$$

$$R = \begin{bmatrix} \cos A_y & \sin A_y \sin A_x & \sin A_y \cos A_x \\ 0 & \cos A_x & -\sin A_x \\ -\sin A_y & \cos A_y \sin A_x & \cos A_y \cos A_x \end{bmatrix}$$

Now the components of the distance in the perturbed frame are given by v and its representation in the global frame is rotated by the R matrix.

$$v = \begin{bmatrix} d \cos Az \\ d \sin Az \\ 0 \end{bmatrix}$$

$$Rv = \begin{bmatrix} \cos A_y & \sin A_y \sin A_x & \sin A_y \cos A_x \\ 0 & \cos A_x & -\sin A_x \\ -\sin A_y & \cos A_y \sin A_x & \cos A_y \cos A_x \end{bmatrix} \begin{bmatrix} d \cos Az \\ d \sin Az \\ 0 \end{bmatrix}$$

$$Rv = \begin{bmatrix} d(\cos A_y \cos Az + \sin A_y \sin A_x \sin Az) \\ d(\cos A_x \sin Az) \\ d(-\sin A_y \cos Az + \cos A_y \sin A_x \sin Az) \end{bmatrix}$$

The modulus of the X and Y components is Projection value. While the Z value is due to the perturbations, now decoupled.

$$||R|| = d|\cos A_y \cos Az + \sin A_y \sin A_x \sin Az|$$

- ***Euler Angle Conversions***

Euler Angles were converted into quaternions for interfacing with visualisation systems.

- ***Byte-Breakdown, 2's Complement***

Algorithms to break down floating values into a 2 byte array for transmission over serial were created. Also , to regenerate the original number, a 2's complement process was implemented.

- ***BS2 memory computation and Time of Measurement Computations***

These were done to ensure no data leakage occurred and measurements within a resolution of encoder measurements were linearly interpolated

Programs

- **Basic Stamp**

```
'{$STAMP BS2}
'{$PBASIC 2.5}

'----- Variables -----
p0State VAR Bit      ' Current state of P0 (pin10)
prevP0State VAR Bit  ' Previous state of P0
p1State VAR Bit      ' Current state of P1 (pin11)
prevp1State VAR Bit  ' Previous state of P1
p1Counter VAR Byte   ' Counter for P1, starts incrementing from P0 falling edge
angle VAR Word       ' Calculated angle

plCounter VAR Byte
acc_data VAR Byte(12) ' Counter for edges
' Array to hold 12 bytes of data
'----- Initialization -----
prevP0State = 0      ' Assume initial state is HIGH for P0
prevp1State = 0      ' Assume initial state is HIGH for P1
p1Counter = 0        ' Initialize P1 counter to 0
angle = 0            ' Start with angle at 0
plCounter = 0        ' Initialize edge counter

' Variables
Bt1 VAR Byte      ' First byte after header
bt2 VAR Byte      ' Second byte after header
'distance VAR Word
Lidar_pin CON 15
Acc_pin CON 14
PulseSpeed CON 730
BaudMode CON 84

'DEBUG "State Machine Started", CR, CR

DO
```

```

' Read the current state of P0 and P1
p0State = IN12      ' Read the state of pin 10 (P0)
p1State = IN11      ' Read the state of pin 11 (P1)
PULSOUT 13, PulseSpeed
GOSUB P1_Edges      ' Check edges on P1
GOSUB P0_Edges      ' Check edges on P0
PULSOUT 13, PulseSpeed
GOSUB Read_Lidar
PULSOUT 13, PulseSpeed
'DEBUG "STUCK1",CR
GOSUB Accn
' Calculate angle based on states and counter
angle = (180 * p0State) + (45 * plCounter)
'DEBUG HOME
PULSOUT 13, PulseSpeed
'DEBUG "Angle: ", DEC3 angle, " degrees","Distance:", DEC4 distance,"cm , PIN Status: ",BIN1 p0State,BIN1
p1State," Counter:",DEC3 plCounter, CR
GOSUB SEND_data
prevp1State = p1State ' Update previous states
prevP0State = p0State

PULSOUT 13, PulseSpeed      ' Generate pulse on pin 13
PAUSE 1                     ' Small delay for stability (1 ms)
LOOP

' Subroutine to handle edges on P0
P0_Edges:
IF (p0State <> prevP0State) THEN
    plCounter = 0          ' Reset edge counter when P0 changes state
    prevp1State = p1State
ENDIF
RETURN

' Subroutine to handle edges on P1
P1_Edges:
IF (p1State <> prevp1State) THEN
    plCounter = plCounter + 1 ' Increment edge counter when P1 changes state
ENDIF
RETURN

Read_Lidar:
SERIN Lidar_pin, BaudMode, [WAIT($59, $59), Bt1, bt2]' Wait for 0x59, 0x59, then read two bytes
'distance = (bt2 * 256) + Bt1

RETURN

Accn:
SERIN Acc_pin,BaudMode, [STR acc_data\12]
'i VAR Nib
'FOR i=0 TO 11
' acc_data(i)=0
' PAUSE 1
' NEXT
RETURN

SEND_data:
' SEROUT 16, BaudMode,[$59,$59, Bt1,bt2,"YOYO",angle.LOWBYTE,angle.HIGHBYTE,STR acc_data\12,13]
PAUSE 1
'STR acc_data\12, CR
'DEBUG DEC Bt1," ",DEC bt2," d:"," cc: ",DEC ((bt2 * 256) + Bt1) , CR
SEROUT 16, BaudMode, [$59,$59,Bt1,bt2,

```

```

angle.LOWBYTE,angle.HIGHBYTE,
acc_data(0),acc_data(1),acc_data(2),
acc_data(3),acc_data(4),acc_data(5), acc_data(6),
acc_data(7),acc_data(8),acc_data(9),
acc_data(10),acc_data(11)]
RETURN

```

- ROS INTERFACE:

- Launch File:

```

<launch>
  <!-- Load robot description -->
  <env name="LIBGL_ALWAYS_SOFTWARE" value="1" />
  <param name="robot_description" command="cat $(find serial_publisher)/lidar_viz.urdf" />

  <node name="serial_data_publisher" pkg="serial_publisher" type="serial_publisher.py" output="screen" />
    <node name="Lidar_data_publisher" pkg="serial_publisher" type="lidar_publisher.py" output="screen"
  />
  <node name="ODOM_pub" pkg="serial_publisher" type="odom_publisher.py" output="screen" />

  <!-- Launch RViz -->
  <node pkg="rviz" type="rviz" name="rviz" args="-d $(find serial_publisher)/master_rviz_vis.rviz" />
</launch>

```

- URDF:

```

<robot name="LIDAR_SENSOR">
<link name="lidar_viss">
  <visual>
    <origin xyz="0 0 0" rpy="3.14 0.0 0.0"/>
    <geometry>
      <mesh filename="package://serial_publisher/meshes/m1.stl" scale="0.001 0.001 0.001"/>
    </geometry>
  </visual>
</link>
</robot>

```

- Serial Publisher:

```

#!/usr/bin/env python3

import rospy
from std_msgs.msg import String
import serial
import numpy as np
from std_msgs.msg import Float32MultiArray
encoding= 'ascii' #initially this was utf-8
def convert_to_signed(value):
    # Convert a 16-bit unsigned integer to signed (-32,768 to 32,767)
    if value > 0x7FFF: # If the value exceeds the maximum for signed 16-bit integers
        return value - 0x10000
    return value

def Process_and_Package(data):
    # Ensure data is exactly 18 bytes
    data=[89,89] +data
    #print("YOYOYOY data is:",data)
    if len(data) != 18:
        raise ValueError("Data must be exactly 18 bytes long")

    # Check if the first two bytes are 89, 89
    if data[0] != 89 or data[1] != 89:
        raise ValueError("First two bytes must be 89, 89 actual:",data)

    # Calculate distance from byte 3 and byte 4

```

```

distance = (data[3] * 256) + data[2]

# Extract angle from byte 5
angle = data[4] + data[5]*256

# Calculate angleX, angleY, angleZ, accX, accY, accZ
angleX = convert_to_signed(data[6] + (data[7] <<8)) / 100.0
# Calculate angleY, angleZ, accX, accY, accZ normally
angleY = convert_to_signed(data[8] + (data[9] <<8)) / 100.0
angleZ = convert_to_signed(data[10] + (data[11] <<8)) / 100.0
accX = convert_to_signed(data[12] + (data[13] <<8)) / 100.0
accY = convert_to_signed(data[14] + (data[15] <<8)) / 100.0
accZ = convert_to_signed(data[16] + (data[17] <<8)) / 100.0
# Return results in a list

return [distance,angle,angleX,angleY, angleZ, accX,accY,accZ]

def serial_publisher():
    # Initialize the ROS node
    rospy.init_node('serial_publisher', anonymous=True)

    # Create a publisher on the 'serial_data' topic
    pub = rospy.Publisher('serial_data', Float32MultiArray, queue_size=2)
    #NOTEOTEOT :: EDIT QUEUE TO ADJUST LAG

    # Set the loop rate (10 Hz)
    rate = rospy.Rate(100)

    # Open the serial port (adjust '/dev/ttyACM0' and baudrate as needed)
    try:
        ser = serial.Serial('/dev/ttyUSB0', 9600, timeout=1)
        rospy.loginfo("Serial port opened successfully")
    except serial.SerialException as e:
        rospy.logerr(f"Failed to open serial port: {e}")
        return

    while not rospy.is_shutdown():
        try:
            # Read data from the serial port
            t=ser.read(1)
            #rospy.loginfo(f"Raw first: {t}")
            if t==b'Y':
                #rospy.loginfo("READ FIRST!")
                t==ser.read(1)
                if t==b'Y':
                    #rospy.loginfo("READ Second!")
                    line = ser.read(16)
                    integers = ([byte for byte in line])
                    #rospy.loginfo(f"RAW: {line}")
                    rospy.loginfo(f"Read from serial:
{Process_and_Package(integers)}")

                    msg = Float32MultiArray()
                    msg.data = Process_and_Package(integers)
                    # Publish the data to the topic
                    pub.publish(msg)

        except Exception as e:
            rospy.logerr(f"Error reading from serial port: {e}")

        rate.sleep()

if __name__ == '__main__':

```

```

try:
    serial_publisher()
except rospy.ROSInterruptException:
    pass

```

- Odom Publisher:
#!/usr/bin/env python3

```

import rospy
import tf
from tf2_ros import TransformBroadcaster
from nav_msgs.msg import Odometry
from geometry_msgs.msg import Quaternion
from std_msgs.msg import Float32MultiArray
from geometry_msgs.msg import TransformStamped
import numpy as np
import time
import math

# Initialize state variables
x = np.zeros((6,)) # Initial state vector: [x, y, vx, vy, ax, ay]
P = np.eye(6) # Initial covariance matrix
F = np.eye(6) # State transition matrix (to be updated dynamically)
Q = np.eye(6) * 0.1 # Process noise covariance
R = np.eye(2) * 0.1 # Measurement noise covariance
H = np.array([[0, 0, 0, 0, 1, 0], [0, 0, 0, 0, 0, 1]]) # Observation matrix

last_time = None

def euler_to_quaternion(roll, pitch, yaw):
    """
    Convert Euler angles (roll, pitch, yaw) to a quaternion.
    """
    q = tf.transformations.quaternion_from_euler(roll, pitch, yaw)
    return Quaternion(*q)

def serial_data_callback(msg):
    """
    Callback function to process the serial_data topic and publish odometry.
    """
    # Extract required data from the message
    angle_x = msg.data[2]
    angle_y = msg.data[3]
    angle_z = msg.data[4]
    acc_x = msg.data[5]
    acc_y = msg.data[6]
    acc_z = msg.data[7]

    # Compute velocity from acceleration (assuming constant time step for simplicity)
    #global x, y, th, last_time
    global x, P, F, last_time, old_angle, ang_Zz
    current_time = rospy.Time.now()

    dt = (current_time - last_time).to_sec()
    last_time = current_time

    ang_Z = -1 * math.radians(msg.data[1])
    if ang_Z != old_angle:
        ang_Zz = -1 * math.radians(msg.data[1])
    else:

```

```

        ang_Zz=ang_Zz - math.radians(6)
    old_angle=        ang_Z
    ang_Zz=0
    #vx = acc_x * dt
    #vy = acc_y * dt
    #vth = acc_z * dt

    # Update position based on velocity
    #x += vx * dt
    #y += vy * dt
    #th += vth * dt

    #KALMAN IMPLEMENTATION
    F[0][2] = F[1][3] = dt
    F[2][4] = F[3][5] = dt
    # Prediction step
    x_pred = F @ x
    P_pred = F @ P @ F.T + Q
    # Update step using accelerometer data as measurement
    z = np.array([acc_x, acc_y]) # ax and ay from accelerometer
    y = z - H @ x_pred
    S = H @ P_pred @ H.T + R
    K = P_pred @ H.T @ np.linalg.inv(S)
    x = x_pred + K @ y
    P = (np.eye(len(K)) - K @ H) @ P_pred
    print(f"Estimated Position: x={x[0]}, y={x[1]}")
    # Create quaternion from Euler angles

    transform = TransformStamped()
    transform.header.stamp = rospy.Time.now()
    transform.header.frame_id = "world"
    transform.child_frame_id = "lidarddd"#"laser_frame"
    transform.transform.translation.x = 0.0
    transform.transform.translation.y = 0.0
    transform.transform.translation.z = 0.0
    quaternion = euler_to_quaternion(3.14+math.radians(angle_x), math.radians(angle_y), 0)#angle_z)
    transform.transform.rotation = quaternion
    transform2 = TransformStamped()
    transform2.header.stamp = rospy.Time.now()
    transform2.header.frame_id = "world"
    transform2.child_frame_id = "lidar_viss"#"laser_frame"
    transform2.transform.translation.x = 0.0
    transform2.transform.translation.y = 0.0
    transform2.transform.translation.z = 0.0
    quaternion2 = euler_to_quaternion(3.14+math.radians(angle_x), math.radians(angle_y), ang_Zz)#angle_z)
    transform2.transform.rotation = quaternion2
    print(quaternion)
    #transform.transform.rotation.z=0
    #transform.transform.rotation.x = quaternion[0]
    #transform.transform.rotation.y = quaternion[1]
    #transform.transform.rotation.z = quaternion[2]
    #transform.transform.rotation.w = quaternion[3]

    # Publish the transform over tf
    odom_broadcaster.sendTransform(transform)
    odom_broadcaster.sendTransform(transform2)

    # Create and populate the Odometry message
    odom_msg = Odometry()
    odom_msg.header.stamp = current_time
    odom_msg.header.frame_id = "odom"

```

```

# Set position and orientation in the message
odom_msg.pose.pose.position.x = 0#x
odom_msg.pose.pose.position.y = 0#y
odom_msg.pose.pose.position.z = 0.0
odom_msg.pose.pose.orientation = quaternion

# Set velocity in the message
odom_msg.child_frame_id = "lidarddd"#"laser_frame"
odom_msg.twist.twist.linear.x = 0#vx
odom_msg.twist.twist.linear.y = 0#vy
odom_msg.twist.twist.angular.z = 0# EARLIER: vth

# Publish the message
odom_pub.publish(odom_msg)
print(odom_msg)

if __name__ == "__main__":
    rospy.init_node('serial_to_odom', anonymous=True)

    # Initialize global variables for position and time tracking
    #global last_time
    last_time = rospy.Time.now()
    old_angle=None
    ang_Zz=None
    # Create publisher for Odometry messages
    odom_pub = rospy.Publisher("odom", Odometry, queue_size=50)

    # Create TF broadcaster for transform publishing
    odom_broadcaster = TransformBroadcaster()

    # Subscribe to the serial_data topic (assuming it publishes std_msgs/Float32MultiArray)
    rospy.Subscriber("serial_data", Float32MultiArray, serial_data_callback)

    rospy.spin()

```

- Lidar Data Publisher:
#!/usr/bin/env python3

```

import rospy
from std_msgs.msg import Float32MultiArray
from sensor_msgs.msg import LaserScan
import numpy as np

class LaserScanPublisher:
    def __init__(self):
        # Initialize ROS node
        rospy.init_node('laser_scan_publisher', anonymous=True)

        # Subscriber and Publisher
        self.sub = rospy.Subscriber('serial_data', Float32MultiArray, self.serial_data_callback)
        self.pub = rospy.Publisher('/scan', LaserScan, queue_size=10)

        # Data storage
        self.data_array = [] # Stores distance and angle for current range
        self.current_angle_range = None # Tracks the current 45-degree range

    def serial_data_callback(self, msg):
        # Extract distance and angle from incoming message
        distance = msg.data[0] -5 #OFFSET

```



```

angle = msg.data[1]
#print("DATA READ!: ",msg.data)
# Determine the 45-degree range of the current angle
new_angle_range = int(angle // 45) * 45

# If this is a new angle range, process and publish the previous range
if self.current_angle_range is not None and new_angle_range != self.current_angle_range:
    self.process_and_publish()

    # Reset the array for the new angle range
    self.data_array = []

# Update current angle range and append new data
self.current_angle_range = new_angle_range
self.data_array.append((distance, angle))
#print(self.data_array)
def process_and_publish(self):
    if not self.data_array:
        return

    # Prepare LaserScan message
    scan_msg = LaserScan()
    scan_msg.header.stamp = rospy.Time.now()
    scan_msg.header.frame_id = "lidarddd"

    scan_msg.angle_min = np.radians(self.current_angle_range)
    scan_msg.angle_max = np.radians(self.current_angle_range + 45)
    scan_msg.angle_increment = (scan_msg.angle_max - scan_msg.angle_min) / len(self.data_array)
    scan_msg.time_increment = 0.0
    scan_msg.scan_time = 0.1
    scan_msg.range_min = .01#0.05 # Minimum valid range
    scan_msg.range_max = 1000#10.0 # Maximum valid range

    # Linearize distances across the angle range
    distances = [d/100 for d, _ in self.data_array]
    scan_msg.ranges = distances
    self.pub.publish(scan_msg)
    print("PUBLISHED: ",scan_msg)
def run(self):
    rospy.spin()

if __name__ == '__main__':
    try:
        node = LaserScanPublisher()
        node.run()
    except rospy.ROSInterruptException:
        pass

```

References

[1] “Servo dimensions,” Parallax, <https://www.parallax.com/package/servo-dimensions/> (accessed Dec. 15, 2024).

[2] “MakerFocus Lidar Range Finder Sensor Module TF-Luna,” https://www.amazon.com/dp/B088NVX2L7?ref=cm_sw_r_ud_dp_DZPGV6JMN2S6F1FKYVFM&ref_=cm_sw_r_ud_dp_DZPGV6JMN2S6F1FKYVFM&social_share=cm_sw_r_ud_dp_DZPGV6JMN2S6F1FKYVFM&starsLeft=1&skipTwisterOG=1, (accessed Dec. 15, 2024).

[3] Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics* vol. 7, May 2022.